

Process.hpp

```
#ifndef PROCESS_HPP
#define PROCESS_HPP

#include <vector>
#include <string>

// Simulates a single CPU instruction
// In real CPUs: ADD, MOV, LOAD, etc. take different cycles
// Here: simplified to just track duration (1 tick per instruction)
struct Instruction {
    int duration; // How many ticks this instruction takes

    // Constructor with default value of 1 tick
    Instruction(int d = 1) : duration(d) {}
};

// Process lifecycle states (5-state model)
enum class ProcessState {
    CREATED, // Process object exists but hasn't arrived yet
    READY, // In ready queue, waiting for CPU
    RUNNING, // Currently executing on CPU
    BLOCKED, // Waiting for I/O (not used in this simulation)
    TERMINATED // All instructions completed
};

// Represents a single process (program) to be scheduled
class Process {
public:
    // Constructor: Create process with id, arrival time, priority, and instructions
    Process(int id, int arrivalTime, int priority, const std::vector<Instruction>& instructions);

    // Getters for process attributes
    int getId() const; // Process ID (P1, P2, etc.)
    int getArrivalTime() const; // When process arrives (AT)
    int getPriority() const; // Priority value (lower = higher priority)

    // State management
    ProcessState getState() const; // Get current state
    void setState(ProcessState state); // Transition to new state

    // Instruction control methods
    bool hasNextInstruction() const; // Check if more instructions remain
    const Instruction& peekInstruction() const; // View current instruction
    void advanceMock(); // Execute current instruction (increment PC)

    // Progress tracking
    size_t getProgramCounter() const; // Current instruction index (PC)
    size_t getTotalInstructions() const; // Total instructions (burst time)

private:
    int id; // Unique process identifier
    int arrivalTime; // Clock tick when process arrives
    int priority; // Priority for priority scheduling
    std::vector<Instruction> instructions; // All instructions to execute
    size_t programCounter; // Index of next instruction (like PC register)
```

```
    ProcessState state; // Current lifecycle state
};

#endif // PROCESS_HPP
```

Process.cpp

```
#include "Process.hpp"
#include <stdexcept>

// Constructor: Initialize process with all attributes
// Member initializer list is more efficient than assignment in body
// programCounter starts at 0 (first instruction), state starts as CREATED
Process::Process(int id, int arrivalTime, int priority, const std::vector<Instruction>& instructions)
    : id(id), arrivalTime(arrivalTime), priority(priority), instructions(instructions),
      programCounter(0), state(ProcessState::CREATED) {}

// Simple getters - return private member values
int Process::getId() const {
    return id;
}

int Process::getArrivalTime() const {
    return arrivalTime;
}

int Process::getPriority() const {
    return priority;
}

ProcessState Process::getState() const {
    return state;
}

// Setter for state - allows Scheduler/CPU to transition process states
void Process::setState(ProcessState s) {
    state = s;
}

// Check if process has more instructions to execute
// Returns true if PC is within valid range (PC < total instructions)
bool Process::hasNextInstruction() const {
    return programCounter < instructions.size();
}

// Return current instruction without consuming it
// Throws exception if no instructions remain (safety check)
const Instruction& Process::peekInstruction() const {
    if (!hasNextInstruction()) {
        throw std::out_of_range("No more instructions in process");
    }
    return instructions[programCounter];
}

// Simulate executing one instruction by advancing program counter
// CPU calls this after each tick to consume an instruction
// Safety: Only increments if instructions remain
```

```

void Process::advanceMock() {
    if (hasNextInstruction()) {
        programCounter++;
    }
}

// Return current position in instruction sequence
size_t Process::getProgramCounter() const {
    return programCounter;
}

// Return total number of instructions (equals burst time)
size_t Process::getTotalInstructions() const {
    return instructions.size();
}

```

Scheduler.hpp

```

#ifndef SCHEDULER_HPP
#define SCHEDULER_HPP

#include "Process.hpp"
#include <vector>
#include <queue>

using namespace std;

// Abstract base class for all scheduling algorithms
// Defines common interface that all schedulers must implement
class Scheduler {
public:
    // Virtual destructor ensures proper cleanup of derived classes
    virtual ~Scheduler() = default;

    // Add newly arrived process to scheduler's data structure
    // Called by Emulator when process arrival time is reached
    virtual void addProcess(Process* process) = 0;

    // Decide which process should run next (called every tick)
    // Returns nullptr if no process is ready (CPU idles)
    virtual Process* nextProcess() = 0;
};

// FCFS: First Come First Serve (Non-preemptive)
// Simple FIFO queue - processes execute in arrival order
class FCFScheduler : public Scheduler {
public:
    void addProcess(Process* process) override;
    Process* nextProcess() override;

private:
    queue<Process*> readyQueue; // FIFO queue maintains arrival order
};

// SJF: Shortest Job First (Non-preemptive)
// Selects process with shortest total burst time
// Once started, runs to completion
class SJFScheduler : public Scheduler {

```

```

public:
    void addProcess(Process* process) override;
    Process* nextProcess() override;

private:
    vector<Process*> readyList; // Need to search for shortest
    Process* currentProcess = nullptr; // Track current to avoid preemption
};

// Preemptive SJF (SRTF): Shortest Remaining Time First
// Can switch to shorter job when new process arrives
// Always runs process with least remaining work
class PreemptiveSJFScheduler : public Scheduler {
public:
    void addProcess(Process* process) override;
    Process* nextProcess() override;

private:
    vector<Process*> readyList; // Recalculates shortest every tick
    // No currentProcess - always rechecks for shortest
};

// Priority Scheduler (Preemptive)
// Lower priority number = higher priority
// High-priority process can interrupt low-priority one
class PriorityScheduler : public Scheduler {
public:
    void addProcess(Process* process) override;
    Process* nextProcess() override;

private:
    vector<Process*> readyList; // Search for highest priority
    Process* currentProcess = nullptr; // Optimization for same priority
};

// Round Robin: Time-sharing with fixed quantum
// Each process gets equal time slice (2 ticks)
// Fair rotation through circular queue
class RRScheduler : public Scheduler {
public:
    void addProcess(Process* process) override;
    Process* nextProcess() override;

private:
    int quanta = 2; // Time slice per process (2 ticks)
    int currentQuantumUsed = 0; // Track ticks used by current process
    queue<Process*> readyQueue; // FIFO for fair rotation
    Process* currentProcess = nullptr; // Currently running process
};

#endif // SCHEDULER_HPP

```

Scheduler.cpp - Part 1

```

#include "Scheduler.hpp"
#include <algorithm>
#include <limits>

```

```

using namespace std;

// Helper function: Calculate remaining work for a process
// Used by SJF schedulers to find shortest job
// Returns: total_instructions - program_counter
static size_t remainingInstructions(Process* p) {
    if (!p) return 0; // Null safety
    return p->getTotalInstructions() - p->getProgramCounter();
}

// ===== FCFS SCHEDULER (Non-preemptive) =====
// First Come First Serve: Processes execute in arrival order
// Uses FIFO queue - simplest scheduling algorithm

void FCFSScheduler::addProcess(Process* process) {
    process->setState(ProcessState::READY); // Transition to ready state
    readyQueue.push(process);              // Add to back of queue
}

Process* FCFSScheduler::nextProcess() {
    // Check if queue is empty (no processes ready)
    if (readyQueue.empty()) {
        return nullptr; // CPU will idle
    }

    // Cleanup: Remove any terminated processes from front
    while (!readyQueue.empty() && readyQueue.front()->getState() == ProcessState::
        TERMINATED) {
        readyQueue.pop();
    }

    // After cleanup, check if queue became empty
    if (readyQueue.empty()) {
        return nullptr;
    }

    Process* p = readyQueue.front(); // Get process at front (don't remove yet)

    // If front process has finished all instructions, remove it and recurse
    if (!p->hasNextInstruction()) {
        readyQueue.pop();
        return nextProcess(); // Try next process in queue
    }

    // Return the process at front (stays in queue until done)
    return p;
}

// ===== SJF SCHEDULER (Non-preemptive) =====
// Shortest Job First: Selects process with shortest burst time
// Once started, runs to completion (non-preemptive)

void SJFScheduler::addProcess(Process* process) {
    process->setState(ProcessState::READY);
    readyList.push_back(process); // Add to list (order doesn't matter)
}

```

```

Process* SJFScheduler::nextProcess() {
    // Erase-remove idiom: Remove terminated processes from vector
    // remove_if moves terminated to end, erase actually removes them
    readyList.erase(
        remove_if(readyList.begin(), readyList.end(),
            [](Process* p){ return p->getState() == ProcessState::TERMINATED; }),
        readyList.end()
    );

    // NON-PREEMPTIVE: If current process is still valid, continue with it
    // Don't switch even if shorter job arrives - this is key SJF behavior
    if (currentProcess && currentProcess->getState() != ProcessState::TERMINATED
        && currentProcess->hasNextInstruction()) {
        return currentProcess;
    }

    // Current finished or no current - find shortest job
    Process* best = nullptr;
    size_t bestRem = numeric_limits<size_t>::max(); // Start with max value

    // Linear search through all ready processes
    for (auto* p : readyList) {
        if (p->getState() == ProcessState::READY && p->hasNextInstruction()) {
            size_t rem = remainingInstructions(p);
            if (rem < bestRem) { // Found shorter job
                bestRem = rem;
                best = p;
            }
        }
    }

    currentProcess = best; // Remember this choice for next tick
    return currentProcess; // May be nullptr if no ready processes
}

```

Scheduler.cpp - Part 2

```

// ===== PREEMPTIVE SJF / SRTF SCHEDULER =====
// Shortest Remaining Time First: Always runs process with least work remaining
// PREEMPTIVE: Can switch to shorter job when new process arrives
// Recalculates shortest every tick - no memory of previous choice

void PreemptiveSJFScheduler::addProcess(Process* process) {
    process->setState(ProcessState::READY);
    readyList.push_back(process);
}

Process* PreemptiveSJFScheduler::nextProcess() {
    // Remove completed processes from list
    readyList.erase(
        remove_if(readyList.begin(), readyList.end(),
            [](Process* p){ return p->getState() == ProcessState::TERMINATED; }),
        readyList.end()
    );

    // Find process with absolute minimum remaining time
    Process* best = nullptr;

```

```

size_t bestRem = numeric_limits<size_t>::max();

for (auto* p : readyList) {
    // KEY DIFFERENCE: Check READY *OR* RUNNING (enables preemption)
    // This means currently running process is also considered
    if ((p->getState() == ProcessState::READY || p->getState() == ProcessState::
        RUNNING)
        && p->hasNextInstruction()) {
        size_t rem = remainingInstructions(p);
        if (rem < bestRem) {
            bestRem = rem;
            best = p;
        }
    }
}

// Return shortest - might be different from last tick (preemption!)
return best;
}

// ===== PRIORITY SCHEDULER (Preemptive) =====
// Priority-based: Lower priority number = Higher priority
// Example: P2(priority=1) can preempt P1(priority=2)
// PREEMPTIVE: High-priority process interrupts low-priority one

void PriorityScheduler::addProcess(Process* process) {
    process->setState(ProcessState::READY);
    readyList.push_back(process);
}

Process* PriorityScheduler::nextProcess() {
    // Clean up terminated processes
    readyList.erase(
        remove_if(readyList.begin(), readyList.end(),
            [](Process* p){ return p->getState() == ProcessState::TERMINATED; }),
        readyList.end()
    );

    // Find process with highest priority (smallest priority number)
    Process* best = nullptr;
    int bestPrio = numeric_limits<int>::max(); // Start with worst priority

    for (auto* p : readyList) {
        // Check both READY and RUNNING states (preemptive)
        if ((p->getState() == ProcessState::READY || p->getState() == ProcessState::
            RUNNING)
            && p->hasNextInstruction()) {
            int pr = p->getPriority();
            if (pr < bestPrio) { // Lower number = higher priority
                bestPrio = pr;
                best = p;
            }
        }
    }

    // Fallback: If no process found in list but current is valid, continue
    // Edge case handling for when readyList might be temporarily empty

```

```

if (!best && currentProcess && currentProcess->getState() != ProcessState::TERMINATED
    && currentProcess->hasNextInstruction()) {
    return currentProcess;
}

currentProcess = best; // Remember choice (optimization)
return currentProcess;
}

```

Scheduler.cpp - Part 3

```

// ===== ROUND ROBIN SCHEDULER =====
// Time-sharing algorithm: Each process gets fixed time slice (quantum = 2)
// Fair rotation: After quantum expires, process goes to back of queue
// Prevents starvation: Every process gets CPU time periodically

void RRScheduler::addProcess(Process* process) {
    process->setState(ProcessState::READY);
    readyQueue.push(process); // New arrivals join end of queue
}

Process* RRScheduler::nextProcess() {
    // Cleanup: Remove terminated processes from front of queue
    while (!readyQueue.empty() && readyQueue.front()->getState() == ProcessState::
        TERMINATED) {
        readyQueue.pop();
    }

    // CASE 1: Need to pick a new process
    // Happens when: no current, current terminated, or current finished
    if (currentProcess == nullptr || currentProcess->getState() == ProcessState::
        TERMINATED
        || !currentProcess->hasNextInstruction()) {

        currentQuantumUsed = 0; // Reset quantum counter for new process

        if (readyQueue.empty()) {
            currentProcess = nullptr;
            return nullptr; // No processes available, CPU idles
        }

        // Pop process from front of queue and give it CPU
        currentProcess = readyQueue.front();
        readyQueue.pop();
        return currentProcess;
    }

    // CASE 2: Current process used up its time quantum (2 ticks)
    // Time to rotate! This is the "round robin" behavior
    if (currentQuantumUsed >= quanta) {

        // If current still has work, send it to back of queue
        if (currentProcess->hasNextInstruction()
            && currentProcess->getState() != ProcessState::TERMINATED) {
            currentProcess->setState(ProcessState::READY); // Back to waiting
            readyQueue.push(currentProcess); // Join end of queue (fair rotation)
        }
    }
}

```

```

// Reset quantum for next process
currentQuantumUsed = 0;

// Clean terminated from front
while (!readyQueue.empty() && readyQueue.front()->getState() == ProcessState::
    TERMINATED) {
    readyQueue.pop();
}

// If no other processes waiting, no one to switch to
if (readyQueue.empty()) {
    currentProcess = nullptr;
    return nullptr;
}

// Give CPU to next process in queue
currentProcess = readyQueue.front();
readyQueue.pop();
return currentProcess;
}

// CASE 3: Continue with current process (quantum not expired)
// Increment quantum counter each tick
// Example: quantum=0 -> 1 -> 2, then rotation happens
currentQuantumUsed++;
return currentProcess;
}

```

Operating Systems - Deadlock & Synchronization

Banker's Algorithm (Deadlock Avoidance)

```

#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>
#define N 5 // Number of Processes
#define M 3 // Number of Resource Types

typedef struct {
    int allocation[N][M];
    int max[N][M];
    int available[M];
    int need[N][M];
} SystemState;

int count = 0;

int *isSafe(SystemState *state) {
    // 1. Calculate need matrix
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < M; j++) {
            state->need[i][j] = state->max[i][j] - state->allocation[i][j];
        }
    }

    // 2. Initialize finish array to false
    bool finish[N];
    for (int i = 0; i < N; i++) finish[i] = false;

    // 3. Initialize work array with available

```

```

int work[M];
for (int j = 0; j < M; j++) work[j] = state->available[j];

int *safeSeq = (int *)malloc(N * sizeof(int));
count = 0;

// 4. Find safe sequence
bool found = true;
while (found) {
    found = false;
    for (int i = 0; i < N; i++) {
        if (!finish[i]) {
            bool canAllocate = true;
            for (int j = 0; j < M; j++) {
                if (state->need[i][j] > work[j]) {
                    canAllocate = false;
                    break;
                }
            }
            if (canAllocate) {
                for (int j = 0; j < M; j++) {
                    work[j] += state->allocation[i][j];
                }
                finish[i] = true;
                safeSeq[count++] = i;
                found = true;
                break;
            }
        }
    }
}

return safeSeq;
}

int main() {
    SystemState state = {
        .allocation = {{0, 1, 0}, {2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0, 2}},
        .max = {{7, 5, 3}, {3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3}},
        .available = {3, 3, 2}
    };

    int *safeSeq = isSafe(&state);
    if (count == N) {
        printf("System is SAFE.\nSequence: ");
        for (int i = 0; i < N; i++) printf("P%d ", safeSeq[i]);
    } else {
        printf("System is UNSAFE.\nSequence: ");
        for (int i = 0; i < count; i++) printf("P%d ", safeSeq[i]);
    }
    free(safeSeq);
    return 0;
}

```

Deadlock Demo - Simple 2-Thread Example

```

#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;

```

```
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
```

```
void* thread1_routine(void* arg) {
    printf("T1: Trying lock1...\n");
    pthread_mutex_lock(&lock1);
    printf("T1: Got lock1. Sleep...\n");
    sleep(1);
    printf("T1: Trying lock2...\n");
    pthread_mutex_lock(&lock2); // DEADLOCK!
    printf("T1: Got lock2.\n");
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    return NULL;
}
```

```
void* thread2_routine(void* arg) {
    printf("T2: Trying lock2...\n");
    pthread_mutex_lock(&lock2);
    printf("T2: Got lock2. Sleep...\n");
    sleep(1);
    printf("T2: Trying lock1...\n");
    pthread_mutex_lock(&lock1); // DEADLOCK!
    printf("T2: Got lock1.\n");
    pthread_mutex_unlock(&lock1);
    pthread_mutex_unlock(&lock2);
    return NULL;
}
```

```
// Hold-and-Wait Avoidance (acquires all locks at once)
```

```
void* thread1_no_deadlock(void* arg) {
    pthread_mutex_lock(&lock1);
    pthread_mutex_lock(&lock2); // Both at once
    printf("T1: Working with both locks\n");
    sleep(1);
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    return NULL;
}
```

```
void* thread2_no_deadlock(void* arg) {
    pthread_mutex_lock(&lock1);
    pthread_mutex_lock(&lock2); // Both at once
    printf("T2: Working with both locks\n");
    sleep(1);
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    return NULL;
}
```

```
int main() {
    pthread_t t1, t2;
    int choice;
    printf("1. Deadlock 2. No Deadlock\n");
    scanf("%d", &choice);

    if (choice == 1) {
        pthread_create(&t1, NULL, thread1_routine, NULL);
        pthread_create(&t2, NULL, thread2_routine, NULL);
    }
}
```

```
} else {
    pthread_create(&t1, NULL, thread1_no_deadlock, NULL);
    pthread_create(&t2, NULL, thread2_no_deadlock, NULL);
}
pthread_join(t1, NULL);
pthread_join(t2, NULL);
return 0;
}
```

Deadlock Avoidance - Circular Wait Prevention (5 Threads)

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#define THREAD_COUNT 5
```

```
pthread_mutex_t locks[THREAD_COUNT];
```

```
void* worker(void* arg) {
    int idx = *(int *)arg;
    int next_idx = (idx+1)%THREAD_COUNT;
    free(arg);

    // ONE IF STATEMENT: Break circular wait with lock ordering
    if (idx == THREAD_COUNT - 1) {
        // Last thread locks in reverse order
        printf("Thread %d: Locking %d (reverse)\n", idx, next_idx);
        pthread_mutex_lock(&locks[next_idx]);
        sleep(1);
        printf("Thread %d: Trying to lock %d\n", idx, idx);
        pthread_mutex_lock(&locks[idx]);
        printf("Thread %d: Success!\n", idx);
        pthread_mutex_unlock(&locks[idx]);
        pthread_mutex_unlock(&locks[next_idx]);
    } else {
        // Normal order for other threads
        printf("Thread %d: Locking %d\n", idx, idx);
        pthread_mutex_lock(&locks[idx]);
        sleep(1);
        printf("Thread %d: Trying to lock %d\n", idx, next_idx);
        pthread_mutex_lock(&locks[next_idx]);
        printf("Thread %d: Success!\n", idx);
        pthread_mutex_unlock(&locks[idx]);
        pthread_mutex_unlock(&locks[next_idx]);
    }
    return NULL;
}
```

```
int main() {
    pthread_t threads[THREAD_COUNT];
    for (int i = 0; i < THREAD_COUNT; i++)
        pthread_mutex_init(&locks[i], NULL);
    for (int i = 0; i < THREAD_COUNT; i++) {
        int *idx = (int*)malloc(sizeof(int));
        *idx = i;
        pthread_create(&threads[i], NULL, worker, idx);
    }
    for (int i = 0; i < THREAD_COUNT; i++)
```



```

    pthread_join(threads[i], NULL);
    printf("\nAll threads completed - No deadlock!\n");
    for (int i = 0; i < THREAD_COUNT; i++)
        pthread_mutex_destroy(&locks[i]);
    return 0;
}

```

Wait-For Graph Construction (C++)

```

#include <iostream>
#include <vector>
#include <map>
using namespace std;

struct Allocation { int processId; int resourceId; };
struct Request { int processId; int resourceId; };

class WaitForGraph {
private:
    int numProcesses;
    map<int, vector<int>> adjList;
public:
    WaitForGraph(int n) : numProcesses(n) {}

    void addEdge(int from, int to) {
        adjList[from].push_back(to);
    }

    void display() {
        cout << "\nWait-For Graph:" << endl;
        for (int i = 0; i < numProcesses; i++) {
            cout << "P" << i << " -> ";
            if (adjList.find(i) != adjList.end() && !adjList[i].empty()) {
                for (size_t j = 0; j < adjList[i].size(); j++) {
                    cout << "P" << adjList[i][j];
                    if (j < adjList[i].size() - 1) cout << ", ";
                }
            } else { cout << "none"; }
            cout << endl;
        }
    }
    map<int, vector<int>>& getAdjList() { return adjList; }
};

```

```

class ResourceGraph {
private:
    int numProcesses, numResources;
    vector<Allocation> allocations;
    vector<Request> requests;
public:
    ResourceGraph(int p, int r) : numProcesses(p), numResources(r) {}

    void addAllocation(int process, int resource) {
        allocations.push_back({process, resource});
    }
    void addRequest(int process, int resource) {
        requests.push_back({process, resource});
    }
}

```

```

WaitForGraph* buildWaitForGraph() {
    WaitForGraph* wfg = new WaitForGraph(numProcesses);

    // For each process requesting a resource
    for (const auto& req : requests) {
        int requestingProc = req.processId;
        int requestedRes = req.resourceId;

        // Find which process holds this resource
        for (const auto& alloc : allocations) {
            if (alloc.resourceId == requestedRes &&
                alloc.processId != requestingProc) {
                wfg->addEdge(requestingProc, alloc.processId);
            }
        }
    }
    return wfg;
};

// Example: P0 holds R0, wants R1; P1 holds R1, wants R0 (DEADLOCK!)
int main() {
    ResourceGraph rag(2, 2);
    rag.addAllocation(0, 0); // P0 holds R0
    rag.addAllocation(1, 1); // P1 holds R1
    rag.addRequest(0, 1);    // P0 requests R1
    rag.addRequest(1, 0);    // P1 requests R0

    WaitForGraph* wfg = rag.buildWaitForGraph();
    wfg->display(); // Shows: P0->P1, P1->P0 (cycle!)
    delete wfg;
    return 0;
}

```

Cycle Detection for Deadlock (C++)

```

#include <iostream>
#include <vector>
#include <map>
using namespace std;

class WaitForGraph {
private:
    int numProcesses;
    map<int, vector<int>> adjList;
public:
    WaitForGraph(int n) : numProcesses(n) {}
    void addEdge(int from, int to) { adjList[from].push_back(to); }

    // DFS-Based Cycle Detection with Recursion Stack
    bool DFS_CycleUtil(int v, vector<bool>& vis, vector<bool>& recStack,
                      vector<int>& path) {
        vis[v] = true;
        recStack[v] = true;
        path.push_back(v);

        if (adjList.find(v) != adjList.end()) {
            for (int neighbor : adjList[v]) {
                if (!vis[neighbor]) {

```

```

        if (DFS_CycleUtil(neighbor, vis, recStack, path))
            return true;
    } else if (recStack[neighbor]) { // CYCLE FOUND!
        path.push_back(neighbor);
        return true;
    }
}
}
recStack[v] = false;
path.pop_back();
return false;
}

bool detectCycle_DFS(vector<int>& cycle) {
    vector<bool> vis(numProcesses, false);
    vector<bool> recStack(numProcesses, false);
    vector<int> path;

    for (int i = 0; i < numProcesses; i++) {
        if (!vis[i]) {
            if (DFS_CycleUtil(i, vis, recStack, path)) {
                int start = path.back();
                cycle.clear();
                bool inCycle = false;
                for (int v : path) {
                    if (v == start) inCycle = true;
                    if (inCycle) cycle.push_back(v);
                }
                return true;
            }
        }
    }
    return false;
}

int main() {
    WaitForGraph wfg(3);
    wfg.addEdge(0, 1); wfg.addEdge(1, 2); wfg.addEdge(2, 0);

    vector<int> cycle;
    if (wfg.detectCycle_DFS(cycle)) {
        cout << "Deadlock detected! Cycle: ";
        for (size_t i = 0; i < cycle.size(); i++)
            cout << "P" << cycle[i] << (i < cycle.size()-1 ? "->" : "\n");
    } else {
        cout << "No deadlock (Safe state)" << endl;
    }
    return 0;
}

```

Key Concepts

```

// Deadlock Conditions (ALL 4 must hold):
// 1. Mutual Exclusion
// 2. Hold and Wait
// 3. No Preemption
// 4. Circular Wait

```

```

// Banker's Algorithm:
// - Need[i][j] = Max[i][j] - Allocation[i][j]
// - Work[j] = Available[j]
// - Find process i: Need[i] <= Work AND finish[i] = false
// - Work = Work + Allocation[i]
// - Safe if all processes finish

// Cycle Detection:
// - Wait-For Graph: Process -> Process dependencies
// - DFS with recursion stack detects cycles
// - O(V+E) time complexity
// - If cycle exists -> Deadlock exists

// Prevention Strategies:
// - Lock Ordering (break circular wait)
// - Hold-and-Wait avoidance
// - Timeouts and retry
// - Banker's algorithm (avoidance)

```

Compilation & Execution Commands

```

// ===== CPU SCHEDULING (C++) =====
// Compile all files together:
g++ -std=c++17 Process.cpp Scheduler.cpp main.cpp -o scheduler
./scheduler

// ===== BANKER'S ALGORITHM (C) =====
gcc banker_algorithm.c -o banker
./banker

// ===== DEADLOCK DEMO 2-THREAD (C) =====
gcc deadlock_demo.c -o deadlock_demo -pthread
./deadlock_demo
// Input: 1 (for deadlock) or 2 (for no deadlock)

// ===== DEADLOCK CIRCULAR WAIT 5-THREAD (C) =====
gcc deadlock_circular.c -o deadlock_circular -pthread
./deadlock_circular

// ===== CYCLE DETECTION (C++) =====
g++ -std=c++17 cycle_detection.cpp -o cycle
./cycle

// ===== WAIT-FOR GRAPH (C++) =====
g++ -std=c++17 wait_for_graph.cpp -o wfg
./wfg

// Common flags:
// -std=c++17 : Use C++17 standard
// -pthread   : Enable POSIX threads
// -o filename : Output executable name
// -Wall      : Show all warnings (optional, good practice)
// -g         : Include debug symbols (optional)

// Windows (MinGW/MSYS2):
g++ Process.cpp Scheduler.cpp main.cpp -o scheduler.exe
gcc banker.c -o banker.exe

// Run with input redirection:

```



```
./scheduler < input.txt  
./banker < test_case.txt
```